Code Explanation

AWS Glue PySpark ETL Job - Code ExplanationThis code implements a complete customer order analytics pipeline using AWS Glue and PySpark. Here's a clear step-by-step breakdown:---

1. Configuration Management

Function: `get_job_parameters()`- Attempts to load job configuration from a YAML file (`config/glue_params.yaml`)- If the file is missing or unreadable, returns hardcoded default parameters- Parameters include: - S3 paths for input data (customer and order CSV files) - S3 paths for output data (curated, state, analytics layers) - Glue Data Catalog settings (database and table names) - Hudi configuration for SCD Type 2 implementation - Feature flags (enable_scd2, enable_incremental_processing, enable_data_quality_checks) - CSV parsing options and output format settings---

2. Spark Context Initialization

Function: `initialize_spark_contexts()`- Checks if Spark is already running (useful for testing environments)- If not, initializes AWS Glue-specific contexts: - **SparkContext**: Core Spark functionality - **GlueContext**: AWS Glue-specific operations - **Job**: Glue job tracking and bookmarking- Retrieves the job name from command-line arguments or uses a default- Returns all three contexts for use throughout the pipeline---

3. Data Reading Operations

Class: `DataReader`**Method: `read_csv()`**- Validates the S3 path format (must start with `s3://` or `s3a://`)- Reads CSV files from S3 with configurable options: - Header presence - Schema inference - Custom separator- Normalizes all column names to lowercase for consistency- Returns a Spark DataFrame or None if reading fails- Logs the number of records read---

4. Data Writing Operations

Class: `DataWriter`**Method: `write_dataframe()`**- Validates S3 path and checks for empty DataFrames- Supports two output formats: - **Parquet**: Standard columnar format with compression - **Hudi**: For SCD Type 2 with upsert capabilities- Returns success/failure status**Method: `write_to_catalog()`**- Writes DataFrame to AWS Glue Data Catalog- Converts Spark DataFrame to Glue DynamicFrame- Falls back to S3 write if catalog write fails- Enables data discovery and querying via Athena---

5. Data Quality Operations

Function: `clean_dataframe()`Performs data quality checks and cleaning:1. Replaces string literal "Null" with actual NULL values2. Drops rows containing any NULL values3. Removes duplicate records4. Logs before/after record counts---

6. SCD Type 2 Implementation

Function: `add_scd2_columns()`Adds Slowly Changing Dimension Type 2 tracking columns:- **isactive**: Boolean flag (True for current records)- **startdate**: Timestamp when record became active- **enddate**: Timestamp when record was superseded (NULL for active records)- **opts**: Operation timestamp for Hudi precombine logic**Function: `detect_customer_changes()`**Implements

incremental processing:- Compares current customer data against baseline- Uses left anti-join to find new or changed records- If no baseline exists, treats all records as new- Returns only changed records to minimize processing---

7. Business Logic Transformations

Function: `create_order_summary()`Creates enriched order data:1. Performs inner join between order and customer DataFrames on `custid`2. Selects relevant columns from both datasets: - Order details: orderid, itemname, priceperunit, qty, date - Customer details: custid, name, emailid, region3. Returns denormalized order summary**Function: `calculate_customer_aggregate_spend()`**Calculates customer spending analytics:1. Computes total spend per order line: `priceperunit × qty`2. Groups by customer name and date3. Aggregates total spending using sum4. Returns customer-date level spending summary---

8. Main ETL Pipeline Orchestration

Function: `main()`Executes the complete pipeline in sequential steps:

Step 1-2: Data Ingestion

- Reads customer CSV data from S3- Reads order CSV data from S3

Step 3-4: Data Cleaning

- Applies quality checks to customer data- Applies quality checks to order data

Step 5: Curated Layer

- Writes cleaned customer data to Glue Catalog- Writes cleaned order data to Glue Catalog- Data is now queryable via Athena

Step 6: SCD Type 2 Baseline (Conditional)

If SCD Type 2 is enabled:1. Attempts to read existing customer baseline2. Detects changed records (if incremental processing enabled)3. Adds SCD Type 2 tracking columns4. Writes to Hudi format with upsert operation5. Maintains historical customer data changes

Step 7: Order Summary Creation

- Joins customer and order data- Creates denormalized order summary- Writes to Glue Catalog for analysis

Step 8: Analytics Aggregation

- Calculates customer aggregate spending- Groups by customer and date- Writes to analytics layer in Glue Catalog

Completion

- Commits the Glue job (updates bookmarks)- Logs completion status---

Key Design Patterns

1. **Error Handling**: All I/O operations return None on failure and log errors2. **Defensive Programming**: Validates inputs before processing3. **Separation of Concerns**: Reader, Writer, and transformation logic are isolated4. **Configuration-Driven**: All paths and settings externalized to YAML5. **Incremental Processing**: Only processes changed data when enabled6. **Data Lineage**: Maintains SCD Type 2 history for customer changes7. **Catalog Integration**: Makes all outputs discoverable via Glue Data Catalog---

Data Flow Summary

```
Raw CSV (S3) → Ingest → Clean (remove nulls/duplicates) → Curated Layer (Glue Catalog) → SCD Type 2 Baseline (Hudi) → Order Summary (joined data) → Analytics Aggregation (customer spend) → Glue Catalog (queryable via Athena)
```